Lambda expressions

Allow us to write functions in a shorter way.

Before Java 8:

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello");
    }
};
```

Java 8:

```
Runnable r = () -> System.out.println("Hello");
```

2. Functional interfaces

An interface with exactly one abstract method.

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```

Usage:

```
Calculator c = (a, b) -> a + b;
System.out.println(c.add(10, 20));
```

Common functional interfaces:

- Predicate<T> → returns boolean
- Function<T,R> → converts T to R
- Consumer<T> → consumes T, returns nothing
- Supplier<T> → supplies a value

3. Stream API

Used to process collections in a declarative way.

```
List<Integer> nums = Arrays.asList(10, 20, 30, 40);

nums.stream()
    .filter(n -> n > 20)
    .forEach(System.out::println);
```

Output:
30
40

4. Default and static methods in interfaces
Java 8 allows interfaces to have default and static methods.

```
interface Vehicle {
    default void start() {
        System.out.println("Vehicle started");
    }
}
```

5. Optional
Helps deal with potentially null values.

```
Optional<String> name = Optional.ofNullable(null);

System.out.println(name.orElse("Unknown"));
```

Output:
Unknown

6. New Date/Time API
Java 8 introduced:
LocalDate
LocalTime
LocalDateTime

ZonedDateTime

Example:

```
LocalDate today = LocalDate.now();
```

7. Method references

Instead of:

```
list.forEach(x -> System.out.println(x));
```

we can write:

```
list.forEach(System.out::println);
```

Interview answer

Java 8 introduced Lambda expressions, Functional Interfaces, Stream API, Optional, default and static interface methods, Method References, and the new Date-Time API.

2. Difference between map() and flatMap()

This is a very common Stream API interview question.

map()

map() transforms each element into another element.

Example:

```
List<String> names = Arrays.asList("Ram", "John", "Sam");

List<Integer> lengths = names.stream()
        .map(String::length)
        .collect(Collectors.toList());
```

Result:

```
[3, 4, 3]
```

Think:

A → B

flatMap()

flatMap() is used when each element produces a collection/stream and we want to flatten them into one stream.

Example:

```
List<List<Integer>> numbers = Arrays.asList(
    Arrays.asList(1, 2),
    Arrays.asList(3, 4),
    Arrays.asList(5, 6)
);
```

Using flatMap():

```
List<Integer> result = numbers.stream()
        .flatMap(List::stream)
        .collect(Collectors.toList());
```

Result:

[1, 2, 3, 4, 5, 6]

Simple difference

map:

[1,2] [3,4] [5,6]

↓

Stream<List<Integer>>

flatMap:

[1,2] [3,4] [5,6]

↓

1 2 3 4 5 6

Interview line

map() performs one-to-one transformation, whereas flatMap() transforms and flattens nested structures into a single stream.

3. Intermediate vs Terminal operations in Stream API

Streams have two major types of operations.

Intermediate operations

They transform a stream and return another stream.

Examples:

filter()

map()

flatMap()

sorted()

distinct()

limit()

skip()

Example:

```
stream
    .filter(x -> x > 10)
    .map(x -> x * 2);
```

These operations are generally lazy.

Nothing actually happens until a terminal operation is called.

Terminal operations

They produce a final result or side effect.

Examples:

collect()

forEach()

reduce()

count()

min()

max()

anyMatch()

allMatch()

findFirst()

Example:

```
List<Integer> result = numbers.stream()
        .filter(x -> x > 10)
        .collect(Collectors.toList());
```

Here:

filter() → intermediate

collect() → terminal

Important interview point

```
numbers.stream()
        .filter(x -> x > 10)
        .map(x -> x * 2);
```

This may produce no visible result, because there is no terminal operation.

4. Functional Interface vs Marker Interface

Functional Interface

An interface having exactly one abstract method.

```
@FunctionalInterface
interface MyInterface {
    void show();
}
```

It can be used with lambda expressions:

```
MyInterface obj = () -> System.out.println("Hello");
```

Examples:
Runnable
Callable
Comparator
Predicate
Function
Consumer
Supplier

Note: default and static methods do not count as abstract methods.
Marker Interface
An interface with no methods.

It is used to indicate that a class has some special property/capability.

Example:

```
class Employee implements Serializable {
}
```

Serializable is a classic marker interface.

Other examples include:
Cloneable
Serializable

Difference

Functional Interface	Marker Interface
Exactly one abstract method	No methods
Supports lambda expressions	Used to mark/tag a class
Runnable	Serializable

Predicate Cloneable

5. Can we overload main()? Which one does JVM execute?
Yes, we can overload main().

Example:

```
public class Test {

    public static void main(String[] args) {
        System.out.println("Original main");
    }

    public static void main(int x) {
        System.out.println("Overloaded main");
    }
}
```

The JVM looks for:
public static void main(String[] args)

So it executes:
Original main

It will not automatically execute:
main(int x)

You can call the overloaded version manually:
main(10);

Interview answer
Yes, main() can be overloaded, but the JVM entry point is public static void main(String[]

args).

6. Difference between this and super

this

Refers to the current object.

Example:

```
class Employee {
    String name;

    Employee(String name) {
        this.name = name;
    }
}
```

Here:

this.name

means the current object's name.

this can also call another constructor:

this();

super

Refers to the parent class.

Example:

```
class Animal {
    String name = "Animal";
}

class Dog extends Animal {
```

```
    String name = "Dog";

    void print() {
        System.out.println(this.name);
        System.out.println(super.name);
    }
}
```

Output:
Dog
Animal

super() can call parent constructor

```
class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
}

class Dog extends Animal {
    Dog() {
        super();
    }
}
```

Simple difference
this → current class/object
super → parent class

7. How do you create and handle custom exceptions?
You create a custom exception by extending Exception or RuntimeException.
Checked custom exception

```
class InvalidAgeException extends Exception {

    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Use it:

```
class Test {

    static void checkAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above");
        }
    }

    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output:

```
Age must be 18 or above
```

Runtime exception

```
class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String message) {
        super(message);
```

```
    }
}
```

Difference

Exception

↓

Checked exception

RuntimeException

↓

Unchecked exception

8. Difference between Runnable and Callable

Both are used for tasks executed by threads/executors.

Runnable

```
Runnable task = () -> {
    System.out.println("Running");
};
```

Important:

void run()

It doesn't return a value and cannot directly throw checked exceptions.

Callable

```
Callable<Integer> task = () -> {
    return 100;
};
```

It has:

V call()

It can:

- return a value
- throw checked exceptions

Example:

```
ExecutorService executor = Executors.newSingleThreadExecutor();

Future<Integer> future = executor.submit(() -> 10 + 20);

System.out.println(future.get());

executor.shutdown();
```

Output:

30

Interview table

Runnable	Callable
run()	call()
No return value	Returns value
Cannot directly throw checked exception	Can throw checked exception
executor.execute() commonly used	executor.submit() commonly used

9. Difference between start() and run()

This is extremely important.

start()

Creates/starts a new thread, and JVM eventually invokes run() on that thread.

```
Thread t = new Thread(() -> {
    System.out.println("Hello");
});

t.start();
```

A separate thread executes the task.

run()

If you directly call:

t.run();

you are simply calling a normal method.

No new thread is created.

Remember:

start()

↓

new thread

↓

run()

Whereas:

run()

↓

normal method call

10. Difference between wait() and sleep()

sleep()

Thread.sleep() pauses the current thread for a specified duration.

Thread.sleep(1000);

The thread sleeps for approximately 1 second.

It does not release monitors/locks that the thread already holds.

wait()

wait() is an Object method used for thread communication.

```
synchronized (obj) {
    obj.wait();
```

}

The thread releases the object's monitor and waits until another thread calls:
obj.notify();

or
obj.notifyAll();

Difference
sleep() wait()
Thread method Object method
Used mainly for delay/timing Used for thread coordination
Does not release monitor Releases the object's monitor
Can be called without synchronized context Must own the object's monitor
Wakes after timeout or interruption Wakes through notification, timeout, or interruption

Interview trap
Don't say "sleep() releases the lock."
It does not release the monitor lock.
11. What is synchronization in Java?
Synchronization controls access to shared resources when multiple threads are involved.

Suppose two threads modify:
int count = 0;

at the same time.

Without proper synchronization, you can get a race condition.

Example:
public synchronized void increment() {

```
    count++;
}
```

Only one thread at a time can execute the synchronized instance method for the same object.

You can also synchronize a block:

```
synchronized (this) {
    count++;
}
```

Why use synchronization?

To provide safe coordination around shared mutable state and prevent race conditions.

Related concepts

- synchronized
- volatile
- Lock
- AtomicInteger
- concurrent collections

12. Explain Java memory areas

The important JVM memory areas are:

```
JVM Memory
│
├── Heap
├── Stack
├── Method Area
└── Program Counter (PC)
```

Heap

Objects are generally created here.

```
Employee e = new Employee();
```

The Employee object is stored on the heap.

Heap is shared among threads.

Garbage Collector manages unreachable objects in the heap.

Stack

Each thread has its own stack.

It contains things such as:

- method call frames
- local variables
- references
- intermediate execution information

Example:

```
void test() {
    int x = 10;
}
```

x is associated with the current method's stack frame.

Method Area

Contains class-level information such as:

- class metadata
- method metadata
- runtime constant pool

In modern HotSpot JVMs, the implementation corresponding to the method area is Metaspace, which replaced the old PermGen in Java 8.

Program Counter (PC)

Each thread has a PC register that tracks the current JVM instruction being executed.

Interview-friendly answer

Heap is shared and stores objects; each thread has its own stack for method frames and local execution data; the method area stores class metadata; and the program counter tracks the current instruction for each thread.

13. What is Garbage Collection? What is the purpose of finalize()?

Garbage Collection

Garbage Collection automatically identifies objects that are no longer reachable and reclaims their memory.

Example:

```
Employee e = new Employee();

e = null;
```

If there are no other references to that object, it can eventually become eligible for garbage collection.

Important

You cannot guarantee exactly when GC will run.

Calling:

```
System.gc();
```

is only a request/hint to the JVM, not a guarantee.

finalize()

Historically, finalize() was intended to allow cleanup before an object was reclaimed.

Example:

```
@Override
protected void finalize() {
    System.out.println("Cleanup");
}
```

However, do not use this in modern Java development.

finalize() was deprecated in Java 9 and subsequently marked for removal because it is

unreliable and has significant problems.

For resource cleanup, prefer:
try-with-resources

Example:

```
try (FileInputStream fis = new FileInputStream("test.txt")) {
    // use file
}
```

The resource is automatically closed.

14. Coding: Minimum-length subarray whose sum equals a target

This question needs one important clarification.

If the array contains positive integers, a sliding-window approach works.

Suppose:
array = [2, 3, 1, 2, 4, 3]
target = 7

Possible subarrays:
[2,3,1,2] = 8
[3,1,2,4] = 10
[4,3] = 7

Answer:
[4,3]
length = 2

Java solution for positive numbers

```
public static int minSubArrayLen(int target, int[] nums) {
```

```
    int left = 0;
    int sum = 0;
    int minLength = Integer.MAX_VALUE;

    for (int right = 0; right < nums.length; right++) {

        sum += nums[right];

        while (sum >= target) {
            minLength = Math.min(minLength, right - left + 1);
            sum -= nums[left];
            left++;
        }
    }

    return minLength == Integer.MAX_VALUE ? 0 : minLength;
}
```

Complexity
Time: O(n)
Space: O(1)

Important interview point
This sliding-window solution assumes the numbers are positive/non-negative in the relevant formulation.

If negative numbers are allowed and you need an exact target sum, the problem changes; a prefix-sum/hash-map approach or other technique may be required depending on the exact requirement.

15. SQL: Rank employees based on experience
Suppose we have:
Employee

---

id

name

experience

We can use:

```
SELECT
    id,
    name,
    experience,
    RANK() OVER (ORDER BY experience DESC) AS employee_rank
FROM Employee;
```

Suppose:

name	experience
A	10
B	8
C	8
D	5

Result:

name	experience	rank
A	10	1
B	8	2
C	8	2
D	5	4

Notice the gap after the tie.

16. Difference between RANK(), DENSE_RANK(), ROW_NUMBER()

This is one of the most important SQL interview questions.

Suppose:

Salary

100

90

90

80

RANK()

RANK() OVER (ORDER BY salary DESC)

Result:

100 → 1

90 → 2

90 → 2

80 → 4

There is a gap.

DENSE_RANK()

DENSE_RANK() OVER (ORDER BY salary DESC)

Result:

100 → 1

90 → 2

90 → 2

80 → 3

No gap after ties.

ROW_NUMBER()

ROW_NUMBER() OVER (ORDER BY salary DESC)

Result:

100 → 1
90 → 2
90 → 3
80 → 4

Every row gets a unique number.
Easy way to remember
RANK → ties + gaps
DENSE_RANK → ties + no gaps
ROW_NUMBER → no ties

17. Explain SQL query execution order
This is a common interview trick because the order you write SQL is not the same as the logical order SQL is processed.

Query:

```
SELECT department, COUNT(*) AS total
FROM Employee
WHERE salary > 50000
GROUP BY department
HAVING COUNT(*) > 5
ORDER BY total DESC
LIMIT 10;
```

Logical processing order is approximately:
1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. DISTINCT
7. ORDER BY

8. LIMIT / OFFSET

Let's understand it

1. FROM

First determine the source data.

FROM Employee

2. WHERE

Filter individual rows.

WHERE salary > 50000

3. GROUP BY

Create groups.

GROUP BY department

4. HAVING

Filter groups.

HAVING COUNT(*) > 5

Difference:

WHERE → filters rows

HAVING → filters groups

5. SELECT

Select/project the required columns.

SELECT department, COUNT(*)

6. DISTINCT

Remove duplicate result rows when specified.

7. ORDER BY

Sort the result.

ORDER BY total DESC

8. LIMIT

Return only the requested number of rows.

☆ Very Important Interview Quick Revision

If you have an interview soon, memorize these:

Java 8

Lambda

Functional Interface

Stream

Optional

Method Reference

Default/Static Interface Methods

New Date-Time API

Stream

map → transform

flatMap → transform + flatten

Intermediate → filter, map, flatMap, sorted

Terminal → collect, forEach, count, reduce

OOP keywords

this → current object

super → parent class

Threads

start() → creates/starts a new thread

run() → normal method call if called directly

sleep() → pause; does NOT release monitor

wait() → releases object's monitor and waits for coordination

Runnable vs Callable

Runnable → run() → no return value

Callable → call() → returns value + checked exceptions possible

JVM memory

Heap → objects

Stack → per-thread method frames/local execution

Method Area → class metadata

PC → current instruction

SQL ranking

RANK() → 1,2,2,4

DENSE_RANK() → 1,2,2,3

ROW_NUMBER() → 1,2,3,4

SQL execution

FROM

↓

WHERE

↓

GROUP BY

↓

HAVING

↓

SELECT

↓

DISTINCT

↓

ORDER BY

↓

LIMIT